

Modular MCP Chat Application

System Architecture, Working Flow, Dependency Reference & Technical Glossary

1. Executive Summary & Objective

The **Modular MCP Chat Application** is an extensible, full-stack AI system designed to transition traditional LLM wrapper applications into modern, tool-augmented **Agentic Architectures**. Built strictly adhering to separation-of-concerns principles, the system decouples client-side user interface rendering from asynchronous API routing, Large Language Model (LLM) orchestration, and Model Context Protocol (MCP) tool execution over standard input/output (stdio) streams.

Layer	Technology	Key Role & Responsibility
Frontend	HTML5 + Tailwind CSS + Vanilla JS	Responsive chat UI, bubble animations, DOM updates, and client-side conversational history tracking.
Backend API	FastAPI + Uvicorn (ASGI)	Non-blocking HTTP routing, CORS middleware, Pydantic validation, and server lifecycle management.
LLM Inference	Google GenAI / OpenAI SDK	Generates natural language responses with automatic tool calling (AFC) and fallback support.
MCP Protocol	Model Context Protocol SDK	Manages dynamic tool discovery, sandbox query execution, and database schema discovery.
Database	SQLite (app_data.db)	Stores persistent relational records (seeded with inventory products and company users).

2. Detailed System Working & Execution Flow

When a user enters a prompt, the system executes a coordinated multi-tier flow across the client, backend, LLM, and MCP subsystems:

- **Step 1 — Input Capture & Client Rendering:** The user submits a prompt via `chat_logic.js`. The UI immediately renders the user bubble (right-aligned), activates the animated loading indicator, disables the input field to prevent duplicate submissions, and packages the prompt along with the preceding conversation history.
- **Step 2 — Asynchronous HTTP Dispatch:** A JSON payload is dispatched via `fetch()` to the FastAPI backend endpoint (`POST /api/chat`).
- **Step 3 — Backend Validation & Orchestration:** FastAPI parses and validates the payload using Pydantic models (`ChatRequest`). It invokes `get_llm_response()` inside `llm_service.py`.
- **Step 4 — Model Tool Assessment:** The LLM client (Google Gemini with Automatic Function Calling or OpenAI GPT-4o-mini) inspects the prompt alongside available MCP tool schemas (`get_current_time`, `list_database_tables`, `query_database`).
- **Step 5 — MCP Tool Invocation:** If the prompt requires live data (e.g., product pricing, employee directories, or timestamps), the LLM issues a structured function call. `mcp_client.py` communicates with the background `mcp_server.py` process over stdio to execute the query against SQLite and returns formatted results.
- **Step 6 — Final Response Synthesis:** The LLM incorporates the retrieved tool output into its context and generates the final natural language answer.

- **Step 7 — UI Update & Context Storage:** The FastAPI backend serializes the reply into ChatResponse. chat_logic.js receives the JSON, appends the AI bubble (left-aligned), appends both turns to client memory, and hides the loading spinner.

3. Python Packages & Dependencies Reference

The project utilizes a minimal, highly optimized dependency set divided into primary top-level libraries, transitive supporting packages, and built-in standard modules:

Package Name	Version / Role	Technical Purpose in Application
fastapi	0.141.1 Web API Framework	Provides asynchronous request routing, automated Pydantic schema validation, and OpenAPI documentation generation.
uvicorn[standard]	0.52.4 ASGI Server	Lightning-fast asynchronous server running on uvloop with watchfiles auto-reloading.
mcp	2.1.1 Protocol SDK	Official Model Context Protocol SDK powering stdio client-server communication and tool registration.
google-genai	2.20.0 LLM Inference	Official Google GenAI SDK running Gemini 3.6 Flash with Automatic Function Calling (AFC) on free-tier quotas.
openai	3.3.1 LLM Failover	Provides seamless automated secondary fallback to GPT-4o-mini if primary provider limits are exceeded.
python-dotenv	1.2.3 Secrets Manager	Securely loads environment keys from backend/.env, ensuring credentials are never exposed in Git.
pydantic	2.13.4 Transitive Dep	Enforces strict data schemas and type checking on all inbound and outbound JSON payloads.
sqlite3	Built-in Module	Zero-configuration relational database engine embedded directly in the MCP server process.

4. Technical Glossary & Key Terminology

A reference guide to the core software engineering and artificial intelligence concepts implemented in this system:

- **Model Context Protocol (MCP):** An open industry protocol created to standardize how Large Language Models connect securely to external tools, databases, and local file systems without proprietary wrappers.
- **Tool / Function Calling:** The capability of an LLM to recognize when it lacks information, generate structured JSON arguments for an external function, and incorporate the returned data into its final response.
- **LLM Statelessness:** The fundamental architecture of language models where each API request is completely independent. The model retains no internal memory of previous requests; historical context must be explicitly supplied in the request body.
- **Multi-Turn Conversational Memory:** The mechanism of maintaining an array of preceding turns ([{'role': 'user', ...}, {'role': 'assistant', ...}]) and forwarding it with each prompt so the AI understands follow-up questions.
- **Sliding Window & Summarization:** Memory optimization strategies for long chats. A sliding window preserves only the most recent N turns, while summarization condenses older turns into a concise paragraph to minimize token costs.
- **ASGI (Asynchronous Server Gateway Interface):** The modern Python standard for asynchronous web servers, enabling applications to handle thousands of concurrent I/O operations (like waiting for database queries or LLM

responses) efficiently.

- **Stdio Transport:** Inter-process communication (IPC) using standard input and output streams. Used by our MCP client to launch and talk to the MCP server subprocess safely.
- **CORS (Cross-Origin Resource Sharing):** A browser security mechanism restricting web pages from making HTTP requests to a different domain or port. Enabled in FastAPI via CORSMiddleware to allow browser-to-backend communication.
- **Graceful Degradation / Resilience:** Fault-tolerant design where the failure of a non-critical component (such as an offline MCP server) does not crash the entire application, but instead results in a clean notification to the user.

Generated for Modular MCP Chat Application Project Documentation & Code Review.